MOBILE APPLICATION AND SERVICE DEVELOPMENT

Lecture 3

Kotlin II · Hello Android · Introduction to Jetpack Compose

Academic year 2026/27 · Faculty of Electrical Engineering, University of Tuzla

Today’s running example: the attendance app’s data model and its first screen

TODAY'S PLAN

Model first, then the screen

PART 1 ·

Kotlin II

- Classes and data classes
- Enum and sealed types
- Interfaces, extensions
- Parsing text
- Collection operations

PART 2 ·

Hello Android

- What an Android app is
- SDK versions as a decision
- Project tour
- First run
- Android in 2026

PART 3 ·

Intro to Compose

- Declarative UI
- Composable functions
- Column, Row, Box

WARM-UP QUIZ · LECTURE 2

What do the three lines print?

```
val nickname: String? = null
val grades = listOf(7, 9, 6, 10)

println(nickname?.length ?: 0)
println(grades.filter { it >= 9 })
println(grades.map { it * 10 }.last())
```

Write your three answers down. One minute.

Then: what does **it** mean inside the braces?

ANSWER

0 · [9, 10] · 100

```
val nickname: String? = null

val grades = listOf(7, 9, 6, 10)



println(nickname?.length ?: 0)
// 0 – safe call gives null, Elvis gives 0

println(grades.filter { it >= 9 })
// [9, 10] – it = one element

println(grades.map { it * 10 }.last())
// 100
```

it is the current element of the list.

Notice: no parentheses around the lambda. We'll see why in two slides.

RECAP · FILTER AND MAP

filter keeps elements. map transforms them

`grades.filter { it >= 9 }`



Lambda returns **Boolean**: keep or drop. Same type, list same size or **smaller**.

`grades.map { it * 10 }`



Lambda returns the **new value**. List **always the same size**, type can change.

```
// chain: filter first, then map
checkIns.filter { it.minute > 615 }.map { it.student.name }
// List<CheckIn> → List<CheckIn> → List<String>: names of latecomers
```

it = the current element. Same as **{ g -> g >= 9 }**.

TRAILING LAMBDA

If the last argument is a lambda, it goes outside the ()

```
val grades = listOf(7, 9, 6, 10)
// 1. lambda as a normal argument
grades.filter({ it >= 9 })

// 2. last argument: move it outside
grades.filter() { it >= 9 }

// 3. only argument: drop the empty ()
grades.filter { it >= 9 }

// with another argument in front
grades.fold(0) { acc, g -> acc + g }
```

All three lines do **exactly the same**. Everyone writes line 3.

Compose is built for this shape:

```
Button(onClick = { checkIn() }) {
    Text("Check in")
}
Column { Text("A"); Text("B") }
```

ON SCREEN



PART 1

Kotlin II: modelling our app

One example all lecture. Every concept adds one piece.

`Student` `CheckIn` `Window` `CheckInStatus` `CheckInRule`

A class bundles data and behaviour

- **Primary constructor** in the header
- **val / var** in it = properties
- **Default values** = optional arguments
- **init** runs when an object is created
- **private** = visible only inside the class
- No **new** keyword

Android: ViewModels and repositories are classes. You will write **private val _state** in week 7.

```
class Lecture(
    val code: String,
    val room: String = "A1"
) {
    private var checkIns = 0

    init { println("Lecture $code created") }

    fun checkIn() { checkIns++ }
    fun count() = checkIns
}

val l3 = Lecture("RMAS-3")  // room = "A1"
l3.checkIn()
// l3.checkIns = 50  // Error: private
```

QUIZ · CLASSES

What does this print, in which order?

```
class Student(val name: String) {
    init { println("1: $name") }
    val upper = name.uppercase()
    init { println("2: $upper") }
}

fun main() {
    println("start")
    Student("Lejla")
}
```

Bonus: what if the second **init** came *before* the line with **upper**?

ANSWER

Top to bottom, in the order written

```
start
1: Lejla
2: LEJLA
```

init blocks and property initializers run **in the order they appear**.

Bonus: **compile error**. upper is used before it is initialized.

2 · DATA CLASSES

A class that just holds data

One word, **data**, and the compiler writes for you:

- **toString()** readable print
- **equals()** compares values
- **copy()** new object, some fields changed
- **destructuring** val (id, name) = s

Android: UI state is a data class. You update it with **state.copy(isLoading = false)**.

```
data class Student(val id: String, val name: String)



data class CheckIn(

    val student: Student,

    val minute: Int    // 10:07 = 607

)



val amar = Student("IB101", "Amar Hodžić")

val first = CheckIn(amar, 607)

val fixed = first.copy(minute = 612)



println(fixed)

// CheckIn(student=Student(id=IB101, ...), minute=612)

val (id, name) = amar
```

QUIZ · DATA CLASSES

true or false? And what is printed?

```
class A(val name: String)
data class B(val name: String)

println(A("Amar") == A("Amar"))  // 1
println(B("Amar") == B("Amar"))  // 2
println(A("Amar"))               // 3
println(B("Amar"))               // 4
```

Four lines, four answers.

ANSWER

false · true · A@1b6d3586 · B(name=Amar)

Normal class

== asks: is it the **same object** in memory? Two separate objects, so false. Printing gives a class name and a hash.

Data class

== compares the **values** of the constructor properties. Same name, so true. Printing shows the values.

Why it matters: Compose compares old and new state to decide what to redraw.

3 · ENUM CLASSES

A fixed set of values

- Only these values exist, no typos possible
- Each value is **one object**, created once
- Values can carry **properties**
- **entries** = all values as a list
- **name**, **ordinal** for free
- Works perfectly with **when**

Android: tabs, filters, sort orders. Our three check-in windows.

```
// The professor opens each window with one tap.
// No fixed times: the enum only names them.
enum class Window(val label: String) {
    START("Start"),
    MIDDLE("Middle"),
    END("End")
}

val w = Window.MIDDLE
println(w.label)          // Middle
println(w.name)           // MIDDLE
println(Window.entries)   // [START, MIDDLE, END]

val hint = when (w) {
    Window.START -> "Check in when you arrive"
    Window.MIDDLE -> "Check in after the break"
    Window.END -> "Check in before you leave"
}
```

QUIZ · ENUM CLASSES

The input comes from a text file

```
println(Window.END.ordinal)         // 1
println(Window.valueOf("MIDDLE"))   // 2
println(Window.valueOf("middle"))   // 3
```

What happens on each line?

Line 3 is what real input looks like. How would you make it safe?

ANSWER

2 · MIDDLE · crash

```
// 3: IllegalArgumentException – names are case-sensitive

// Safe: returns Window? instead of crashing
fun parseWindow(text: String): Window? =
    Window.entries.find {
        it.name.equals(text.trim(), ignoreCase = true)
    }

parseWindow(" middle ")  // MIDDLE
parseWindow("break")     // null
```

ordinal starts at 0.

Rule: never trust input. Return **null** and let the caller decide.

4 · SEALED INTERFACES

A fixed set of cases, each with its own data

- **enum**: every value has the same shape
- **sealed**: Late has minutes, the others don't
- **data object** = one case, no data
- **is** checks the case, then **smart cast**
- No **else**: the compiler checks you covered all

Android: every screen state is **Loading**, **Success(data)** or **Error(message)**.

```
sealed interface CheckInStatus {
    data object OnTime : CheckInStatus
    data class Late(val minutes: Int) : CheckInStatus
    data object Absent : CheckInStatus
}

fun label(s: CheckInStatus): String = when (s) {
    CheckInStatus.OnTime -> "On time"
    is CheckInStatus.Late -> "${s.minutes} min late"
    CheckInStatus.Absent -> "Absent"
}

// s.minutes works: inside that branch
// s is smart-cast to Late
```

QUIZ · SEALED INTERFACES

A new status arrives. What breaks?

```
sealed interface CheckInStatus {
    data object OnTime : CheckInStatus
    data class Late(val minutes: Int) : CheckInStatus
    data object Absent : CheckInStatus
    data class Excused(val reason: String) : CheckInStatus
}

// label() from the previous slide: unchanged
```

A: Nothing, it compiles

B: Crash at runtime

C: Compile error in label()

And if label() had an **else** branch?

ANSWER

C: the compiler stops you, and that's good

```
// Error: 'when' expression must be exhaustive.
// Add the necessary 'is Excused' branch.

is CheckInStatus.Excused -> "Excused: ${s.reason}"

// With else -> "Absent" it compiles...
// ...and every excused student shows as Absent.
```

The compiler lists **every place** you need to update.

else on a sealed type hides bugs. AI adds it a lot.

A contract: what, not how

- **interface** lists functions to provide
- **: CheckInRule** = “I implement it”
- **override** is required
- Code uses the interface, not the class

You’ll see today: **class MainActivity : ComponentActivity()**. With **()** it extends a class; without, it implements an interface.

```
interface CheckInRule {
    fun isValid(c: CheckIn): Boolean
}

class OnTimeRule(val limit: Int) : CheckInRule {
    override fun isValid(c: CheckIn) = c.minute <= limit
}

class RoomRule(val room: String) : CheckInRule {
    override fun isValid(c: CheckIn) = c.room == room
}

val rules: List<CheckInRule> = listOf(OnTimeRule(615), RoomRule("A1"))
val ok = rules.all { it.isValid(checkIn) }
```

QUIZ · INTERFACES

This doesn’t compile. Find two errors

```
class LateRule : CheckInRule() {
    fun isValid(c: CheckIn) = c.minute > 615
}
```

Two errors, two lines.

ANSWER

No () after an interface, and override is required

```
class LateRule : CheckInRule {
    override fun isValid(c: CheckIn) = c.minute > 615
}
```

1. An interface has no constructor, so no **()**.

2. Without **override** the compiler says the function “hides member of supertype”.

DISCUSS IN PAIRS · 2 MINUTES

Both are interfaces. Why is only one sealed?

```
sealed interface CheckInStatus {
    data object OnTime
    data class Late(val minutes: Int)
    data object Absent
}
```

```
interface CheckInRule {
    fun isValid(c: CheckIn): Boolean
}

class OnTimeRule ...
class RoomRule ...
```

And a second one: why is **Window** an enum and not a sealed interface?

SUMMARY · CHOOSING THE TYPE

Values, cases or behaviour?

`enum`

Fixed values, all the same shape

Window: START, MIDDLE, END. Every value has the same fields.

`sealed interface`

Fixed cases, each its own shape

CheckInStatus: OnTime, Late(minutes), Absent. Closed: nobody adds a case elsewhere.

`interface`

A behaviour contract, open

CheckInRule: anything with isValid(). New rules any time, the code using them stays the same.

Ask: is the list closed? Do all cases look alike? Am I describing data or what something does?

6 · EXTENSION FUNCTIONS

Add a function to a type you don’t own

- **fun Type.name()**
- Inside, **this** is the object
- Called like a normal method
- The data keeps the number, **label()** makes the screen text

Compose is built on them: **Modifier.padding()** is an extension function on Modifier.

```
fun Student.initials(): String =
    name.split(" ")
        .map { it.first().uppercaseChar() }
        .joinToString("")

fun CheckInStatus.label(): String = when (this) {
    CheckInStatus.OnTime -> "On time"
    is CheckInStatus.Late -> "$minutes min late"
    CheckInStatus.Absent -> "Absent"
}

amar.initials()                // "AH"
CheckInStatus.Late(5).label()  // "5 min late"
```

QUIZ · EXTENSION FUNCTIONS

Same function, three names

```
Student("1", "lejla kovač").initials()   // 1
Student("2", "Emina").initials()         // 2
Student("3", "Amar  Hodžić").initials()  // 3 (two spaces)
```

One of them crashes. Which one, and why?

ANSWER

"LK" · "E" · crash

```
"Amar  Hodžić".split(" ")
// [Amar, , Hodžić] – an empty string in the middle

"".first()
// NoSuchElementException: Char sequence is empty

// Fix: drop empty parts
name.split(" ").filter { it.isNotBlank() }
```

Works on the examples you tried, crashes on real data.

Next: how to read messy text safely.

7 · PARSING TEXT

From a line of text to an object, or null

lines() splits rows · **split(";")** splits fields · **trim()** removes spaces · **toIntOrNull()** returns Int? · **?: return null** gives up early

```
fun parseTime(t: String): Int? {
    val parts = t.split(":")
    if (parts.size != 2) return null
    val h = parts[0].toIntOrNull()
        ?: return null
    val m = parts[1].toIntOrNull()
        ?: return null
    return h * 60 + m
}
```

```
val log = """
    IB101; Amar Hodžić; 10:07
    IB102; Lejla Kovač; ten past
""".trimIndent()

fun parseLine(line: String): CheckIn? {
    val p = line.split(";").map { it.trim() }
    if (p.size != 3) return null
    val minute = parseTime(p[2]) ?: return null
    return CheckIn(Student(p[0], p[1]), minute)
}

val ok = log.lines().mapNotNull { parseLine(it) }
// 1 check-in: the second line is skipped
```

QUIZ · PARSING

Result, null, or crash?

```
"42".toIntOrNull()          // 1
" 42".toIntOrNull()         // 2
"4.5".toIntOrNull()         // 3
"abc".toInt()               // 4
"a;b;;c".split(";").size    // 5
```

Line 2 surprises most people.

ANSWER

42 · null · null · crash · 4

```
" 42".toIntOrNull()        // null – spaces are not trimmed
" 42".trim().toIntOrNull() // 42

"abc".toInt()
// NumberFormatException

"a;b;;c".split(";")
// [a, b, , c] – empty fields are kept
```

Rule: **trim()** first, then **toIntOrNull()**. Never **toInt()** on input you don’t control.

Answer questions about a list, no loops

```
data class CheckIn(val student: Student, val window: Window, val minute: Int, val status: CheckInStatus)
```

checkIns. …	Returns	Question it answers
filter { it.status is CheckInStatus.Late }	List<CheckIn>	Who was late?
map { it.student.name }	List<String>	Just the names
count { it.window == Window.MIDDLE }	Int	How many in the middle?
sortedBy { it.minute }	List<CheckIn>	Earliest first
maxByOrNull { it.minute }	CheckIn?	Last to arrive
groupBy { it.window }	Map<Window, List<CheckIn>>	Check-ins per window
map { it.minute }.average()	Double	Average arrival

Android: you prepare exactly this kind of list before showing it in a **LazyColumn**.

QUIZ · COLLECTIONS

Nobody checked in at the END window

```
val byWindow = checkIns.groupBy { it.window }

println(byWindow[Window.START]?.size)  // 12
println(byWindow[Window.END])          // ?
```

1. What is the type of byWindow?
2. What does the last line print?
3. Print the count for every window, 0 included.

ANSWER

A missing key is null, not an empty list

```
// 1: Map<Window, List<CheckIn>>
// 2: null – groupBy only creates keys that occur

// 3:
for (w in Window.entries) {
    val n = byWindow[w]?.size ?: 0
    println("${w.label}: $n")
}
// Start: 12 · Middle: 9 · End: 0
```

map[key] always returns a nullable type.

Iterate over **Window.entries** so nothing gets skipped.

CODE REVIEW · IN PAIRS, 5 MINUTES

AI wrote our model. Would you merge it?

```
class CheckIn(var studentId: String, var window: Int,
              var status: String)

fun lateCount(list: List<CheckIn>) =
    list.filter { it.status == "late" }.size

fun describe(c: CheckIn) = when (c.status) {
    "onTime" -> "On time"
    "late" -> "Late"
    else -> "Absent"
}

val a = CheckIn("IB101", 2, "Late")
println(a == CheckIn("IB101", 2, "Late"))
```

It compiles and runs.

Find at least **4 problems**. For each: what goes wrong, and which concept from today fixes it?

SOLUTION

Five problems, today's concepts fix all of them

Problem	Fix
Normal class: == prints false	data class
Everything is var	val
"Late" ≠ "late": count is 0	sealed CheckInStatus
else: a typo becomes Absent	when without else
window = 7 is allowed	enum Window

```
data class CheckIn(
    val studentId: String,
    val window: Window,
    val status: CheckInStatus
)

fun lateCount(list: List<CheckIn>) =
    list.count {
        it.status is CheckInStatus.Late
    }
```

Strings for categories are the most common AI shortcut. The compiler can't help you with them.

BREAK · 10 MINUTES

Android

PART 2

Hello Android

Just enough platform to create, understand and run a first project.

WHAT AN ANDROID APP IS

Four words you'll hear every week

Activity

The entry point the system starts. With Compose, usually one Activity for the whole app.

Manifest

AndroidManifest.xml: the app's ID card. Name, icon, activities, permissions.

Gradle

The build tool. Compiles Kotlin, downloads libraries, packages the app.

APK and AAB

APK installs on a phone. AAB is what you upload to Google Play.

VERSIONS AS A DECISION

Three numbers in every project

`minSdk`

The oldest Android that can install your app. Lower = more phones, more compatibility work.

`targetSdk`

The version you tested against. Google Play sets a minimum every year.

`compileSdk`

The APIs you compile against. Usually the newest.

API level	24	26	29	33	35	36	37
Android	7.0	8.0	10	13	15	16	17

The New Project wizard shows what share of devices each minSdk reaches. Check it there, it changes every year.

QUIZ · VERSIONS

Our attendance app: minSdk 24 or 34?

`minSdk = 24`

Android 7.0 and newer

`minSdk = 34`

Android 14 and newer

Think of every student in this room. What does each choice cost you?

ANSWER

An attendance app must run on every student’s phone

24: almost everyone

Older and budget phones included. Cost: newer APIs need a version check.

34: newest APIs only

Simpler code. Cost: students with older phones can’t check in at all.

In this course: **minSdk 24**, targetSdk and compileSdk = **36**.

PROJECT TOUR · NEW PROJECT → EMPTY ACTIVITY

Where things live

```
app/
  src/main/
    AndroidManifest.xml    // the app's ID card
    java/ba/untz/attendance/
      MainActivity.kt      // entry point
      ui/theme/            // Color, Theme, Type
    res/                    // images, strings, icon
  build.gradle.kts        // SDK versions, libraries
gradle/libs.versions.toml // all library versions
settings.gradle.kts
```

Kotlin code is in the **java/** folder. A historic name.

libs.versions.toml is where you check what AI-generated Gradle code would change.

HELLO WORLD · MAINACTIVITY.KT

You can already read all of this

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            AttendanceTheme {
                Text("Hello RMAS!")
            }
        }
    }
}
```

: ComponentActivity() extends a class

override the system calls onCreate

Bundle? null on the first start

setContent { } a trailing lambda

Text(...) our first composable

Run it on the emulator, then on your phone.

ANDROID IN 2026

Two rules that affect your project

Target API 36

Since 31 August 2026, new apps and updates on Google Play must target **Android 16 (API 36)** or higher.

Every year the minimum moves up. Old tutorials and AI often use old values.

Developer verification

Apps must come from a registered developer. Started 30 Sept 2026 in a few countries, **global in 2027**.

Running from Android Studio over ADB is exempt. To share your APK: a free student account, up to 20 devices.

Sources: developer.android.com · Target API level requirements; Android developer verification FAQ

PART 3

Introduction to Jetpack Compose

Reading: Ghita, Kickstart Modern Android Development, ch. 1, pp. 4–25 · developer.android.com/develop/ui/compose/tutorial

THE PARADIGM SHIFT

Stop changing the screen. Describe it

Imperative: XML views (before)

```
<!-- layout.xml -->
<TextView android:id="@+id/status" />

// MainActivity.kt
val status = findViewById<TextView>(R.id.status)
if (late) status.text = "Late"
else status.text = "On time"
// forget one update: the screen is wrong
```

Declarative: Compose (now)

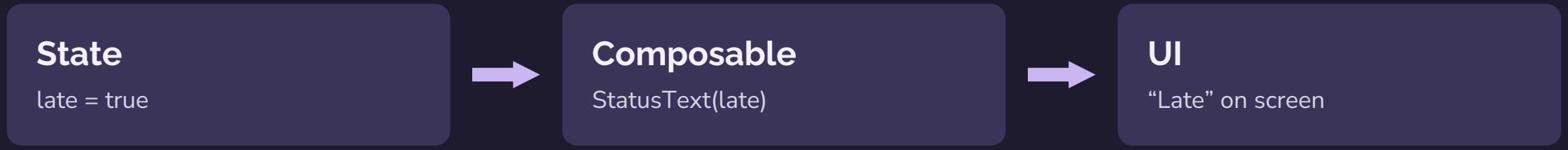
```
@Composable
fun StatusText(late: Boolean) {
    Text(if (late) "Late" else "On time")
}

// one language, no XML, no findViewById
// late changes: the function runs again
```

You describe **what** the UI looks like for given data. Compose works out **how** to update it.

THE CORE IDEA

UI = f(state)



Recomposition: state changes, Compose calls the affected composables again.

Unidirectional data flow: state flows down, events (clicks) flow up.

COMPOSABLE FUNCTIONS

A function that emits UI

- Annotated with **@Composable**
- Returns nothing, it **emits** UI
- Named in **PascalCase**, as a **noun**
- Called only from other composables
- Data comes in as **parameters**

Composition over inheritance: no subclassing Button. Combine small composables into bigger ones.

```
// a normal function returns a value
fun lateCount(list: List<CheckIn>): Int = ...

// a composable function describes UI
@Composable
fun Greeting(name: String) {
    Text("Hello $name!")
}

@Composable
fun StudentHeader(student: Student) {
    Column {
        Greeting(student.name)
        Text(student.id)
    }
}
```

QUIZ · NAMING

Which names follow the rules?

```
// all annotated with @Composable
A  fun CheckInButton(text: String)
B  fun checkInButton(text: String)
C  fun RenderStudentCard(s: Student)
D  fun StudentAvatar(s: Student)
E  fun drawWindowLabel(w: Window)
```

Rule: PascalCase noun, describing what it *is*.

ANSWER

A and D are correct

Name	OK?	Why
CheckInButton	Yes	PascalCase noun
checkInButton	No	camelCase
RenderStudentCard	No	a verb: “render” is what Compose does
StudentAvatar	Yes	PascalCase noun
drawWindowLabel	No	camelCase and a verb

Lint in Android Studio warns about these. AI code often ignores it.

@PREVIEW

See a composable without running the app

- A preview function takes **no parameters**
- It calls your composable with **sample data**
- Several previews = several states at once
- Refreshes in the Design / Split view

A preview is not the app. Always test on a device too.

```
@Preview(showBackground = true, name = "On time")
@Composable
fun StatusTextOnTimePreview() {
    StatusText(late = false)
}

@Preview(showBackground = true, name = "Late")
@Composable
fun StatusTextLatePreview() {
    StatusText(late = true)
}
```

CORE COMPOSABLES

The building blocks you'll use daily

Text

```
Text("Amar Hodžić")
```

Image

```
Image(painterResource(R.drawable.logo), "Logo")
```

Button

```
Button(onClick = { }) { Text("Check in") }
```

Icon

```
Icon(Icons.Default.Check, "Present")
```

Spacer

```
Spacer(Modifier.width(12.dp))
```

TextField · next week

Needs state to remember what you type

LAYOUTS

Column, Row, Box: everything else is built from these

Column

Children **one below another**

```
Column(
  verticalArrangement =
    Arrangement.spacedBy(8.dp)
) {
  Text("Name")
  Text("10:07")
}
```

Row

Children **side by side**

```
Row(
  verticalAlignment =
    Alignment.CenterVertically
) {
  Avatar()
  Text("Amar")
}
```

Box

Children **on top of each other**

```
Box(
  contentAlignment =
    Alignment.Center
) {
  Image(...)
  Text("AH")
}
```

Arrangement = along the main axis. **Alignment** = across it.

COMPOSABLE X-RAY · ON PAPER, 4 MINUTES

Sketch the composable tree of this message



Which Rows, Columns and Boxes?

Which Text and Image?

Draw it as a tree.